

# AI-Powered Candidate Ranking Engine

A Scalable, Memory-Optimized Pipeline for High-Volume Talent Acquisition

Prepared by: **Ravi Raj**  
Role: **AI Systems Engineer / Data Analyst**

# The Challenge & Core Objectives

---

**The Bottleneck:** Processing massive talent pools (100,000+ records) with heavy deep learning models often causes memory crashes or extreme latency on consumer-grade execution setups.

- **Scalability Constraint:** Generating full multi-dimensional text embeddings for high-volume datasets results in severe local resource strain.
- **Hard Targets:** Matching highly specific candidate profiles across complex multiple vectors (Experience, Geography, Role, Engagement).

## The Optimization Goal

Transition a resource-heavy candidate matching layout from an un-executable pipeline bottleneck directly into an active micro-service completed in under 2 minutes.

## Core System Rules

Enforce strict deterministic data constraints, implement zero-loss data streaming, and compute deep semantic similarity mappings seamlessly on common production hardware.

# What We Built

---

An enterprise-grade, fully decoupled modular processing architecture designed for highly efficient script-based pipeline execution.

## **config.py (Infrastructure Configuration)**

Centralizes system path resolutions and handles memory warm-up configurations for the underlying BGE semantic transformer model.

## **scoring.py (Composite Grading Engine)**

Constructs dynamic multidimensional text matrices and runs vectorized signal amplification based on recruiter engagement histories.

## **filters.py (Deterministic Filter Stream)**

Implements memory-safe streaming evaluation criteria to discard out-of-bounds candidate metrics with zero memory allocation overhead.

## **main.py (Pipeline Execution Core)**

Orchestrates data ingestion, runs semantic mappings, applies alphanumeric tie-breakers, and formats outputs strictly into standard compliance tables.

# Architectural Blueprint (How It Works)

---

## 1. JSONL Streaming Generator

Streams data line-by-line from storage, maintaining an  $O(1)$  space complexity footprint and removing large in-memory dataset buffer overheads completely.

## 2. Filter-First Data Compression

Applies strict multi-parameter validation rules (e.g., Experience 5-9 yrs, target geography/relocation flags, non-technical title filtering) to immediately reduce the pool size.

## 3. Dense Semantic Embeddings & Amplification

Maps filtered candidates against the Target Job Description criteria via *BAAI/bge-large-en-v1.5* vectors, applying custom multipliers computed from recruiter response and interview completion trends.

## 4. Tie-Breaker Validation & Final Extraction

Sorts output metrics via an alphanumeric fallback grid to eliminate ranking ambiguity, isolating the absolute Top 100 premium candidates with automated qualitative analysis strings.

# Why We Built It This Way (Design Decisions)

---

- **Filter-First vs. Embed-First Design:** Generating dense vector embeddings across a full raw 100,000 document pool requires intensive GPU setups. Putting lightweight filters *first* compresses the input stream safely, enabling premium deep learning algorithms to run efficiently on local processors.
- **Dynamic Multiplier Mechanics:** Pure resume scanning misses critical performance signals. Our composite matrix uses real-world pipeline engagement history to actively weight responsive talent higher.
- **Decoupled Micro-Scripting Architecture:** Moving from a singular monolithic notebook format into individual domain scripts ensures clean debugging workflows, clear pipeline execution targets, and frictionless integration into web microservices.
- **Deterministic Fallback Handling:** Standard rank functions can produce inconsistent ordering on identical score metrics. Enforcing a strict alphanumeric secondary ID key rule ensures reproducible ranking results.

# Performance Metrics & Final Results

---

**< 2 Min**

## Execution Speed

Processes the entire 100K source profile data stream from end to end effortlessly.

**Stable**

## RAM Utilization

O(1) line streaming architecture eliminates memory growth and OOM crash vulnerabilities.

**Top 100**

## Compliant Export

Delivers structured submission csv output enriched with automated feature text reasons.